Programming Languages 2 - Homework 01

Simon Sutoris 7542170

Task 01

Inference Rules

$n \in \mathbb{N} \land n \in \mathcal{T}$

$\frac{t_1 \in \mathcal{T}, t_2 \in \mathcal{T}}{t_1 + t_2 \in \mathcal{T}}$

$\frac{t_1 \in \mathcal{T}, t_2 \in \mathcal{T}}{t_1 - t_2 \in \mathcal{T}}$

$\frac{t_1 \in \mathcal{T}, t_2 \in \mathcal{T}}{t_1 * t_2 \in \mathcal{T}}$

$\frac{t_1 \in \mathcal{T}, t_2 \in \mathcal{T}, t_3 \in \mathcal{T}}{\texttt{ifz}(t_1; t_2, t_2) \in \mathcal{T}}$

Inductive Definition

$n \in \mathbb{N}$

$$
\begin{array}{ll}
Consts(n) & = \{n\} \\
Consts(t_1 + t_2) & = Consts(t_1) \cup Consts(t_2) \\
Consts(t_1 - t_2) & = Consts(t_1) \cup Consts(t_2) \\
Consts(t_1 * t_2) & = Consts(t_1) \cup Consts(t_2) \\
Consts(\texttt{ifz}(t_1; t_2, t_2)) & = Consts(t_1) \cup Consts(t_2) \cup Consts(t_3)
\end{array}
$$

$$
\begin{array}{ll}
size(n) & = 1 \\
size(t_1 + t_2) & = size(t_1) + size(t_2) + 1 \\
size(t_1 - t_2) & = size(t_1) + size(t_2) + 1 \\
size(t_1 * t_2) & = size(t_1) + size(t_2) + 1 \\
size(\texttt{ifz}(t_1; t_2, t_2)) & = size(t_1) + size(t_2) + size(t_3) + 1
\end{array}
$$

$$
\begin{array}{ll}
depth(n) & = 1 \\
depth(t_1 + t_2) & = \max\{depth(t_1), depth(t_2)\} + 1 \\
depth(t_1 - t_2) & = \max\{depth(t_1), depth(t_2)\} + 1 \\
depth(t_1 * t_2) & = \max\{depth(t_1), depth(t_2)\} + 1 \\
depth(\texttt{ifz}(t_1; t_2, t_2)) & = \max\{depth(t_1), depth(t_2), depth(t_3)\} + 1
\end{array}
$$

Limit of a sequence sets S_n

$$
\begin{array}{lll}
S_0 = & & \\
S_{i+1} = & & \mathbb{N} \\
& \cup & \{t_1 + t_2, t_1 - t_2, t_1 * t_2 \mid t_1, t_2 \in S_i\} \\
& \cup & \{\texttt{ifz}(t_1; t_2, t_2) \mid t_1, t_2, t_3 \in S_i\}
\end{array}
$$

Damit:

$$S = \bigcup_i S_i$$

Task 02

$\texttt{depth}(t)$ ist das kleinste i, sodass $t \in S_i$. Da für jede Tiefe $\text{size}(t)$ um mind. 1 wächst

$$\begin{array}{lll}
\text{size}(n) & := 1 & \Rightarrow +1 \\
\text{size}(t_1 + t_2) & := \text{size}(t_1) + \text{size}(t_2) + 1 & \Rightarrow \geq +3 \\
\text{size}(t_1 - t_2) & := \text{size}(t_1) + \text{size}(t_2) + 1 & \Rightarrow \geq +3 \\
\text{size}(t_1 * t_2) & := \text{size}(t_1) + \text{size}(t_2) + 1 & \Rightarrow \geq +3 \\
\text{size}(\texttt{ifz}(t_1; t_2, t_3)) & := \text{size}(t_1) + \text{size}(t_2) + \text{size}(t_3) + 1 & \Rightarrow \geq +3
\end{array}$$

muss für i Tiefen gelten

$$\text{size}(t) \geq i$$

also

$$\texttt{depth}(t) = i \leq \text{size}(t) \Leftrightarrow \texttt{depth}(t) \leq \text{size}(t)$$

Task 03

Sind die Sprachen die wir angucken fundamental nicht-deterministisch?

Auf S.32 wird `E-IF` definiert, welches die Condition Guard zuerst evaluiert, bevor eines seinter Branches evaluiert wird. Auf S. 40 wird `E-Funny2`, definiert, in welchem der `else` branch evaluiert wird. Damit hat eine Maschine zwei Wege wie es ein `if t1 then t2 else t3` statements evaluiert.

Meine frage: Sind die Sprachen die wir angucken fundamental nicht-deterministsich? Oder ist die Übung nur ein Gedankenexperiment? Das Beispiel ist trivial, aber ich kann mir vortstellen dass in komplexeren Sprachen eine Maschine nicht-deterministische ergebnisse liefern kann.